

Mastering Diverse Domains through World Models

Danijar Hafner^{*12}, Jurgis Pasukonis^{*1}, Jimmy Ba², Timothy Lillicrap¹

¹ Google DeepMind

² University of Toronto

April 3, 2026

Presented by Qin Weng

Model-Free vs. Model-Based RL

Model-Free RL: The Reward-Oriented Approach

- The agent learns a policy or value function directly from trial-and-error interactions with the environment.
- It is strictly focused on maximizing cumulative rewards by learning a direct mapping from states to optimal actions (policy-based) or expected returns (value-based).

Model-Based RL: Learning the Environment Dynamics

- The agent builds a predictive "world model" of the environment and uses this internal simulator to plan actions or train a policy.
- It focuses on learning how the world works (e.g., understanding the underlying physical or logical rules of the environment).

Why MBRL and World Models?

- **Data Efficiency:** MBRL leverages the learned environment model to generate "imagined" rollouts, significantly reducing the sample complexity of real-world interactions.
- **Generalization and Transferability:** MBRL can easily transfer to new tasks within the same world because the learned dynamics of the environment are independent of any specific goal.

*Standard MBRL struggles with high-dimensional inputs. **World Models** overcome this by learning compact **latent representations**.*

From PlaNet to Dreamer

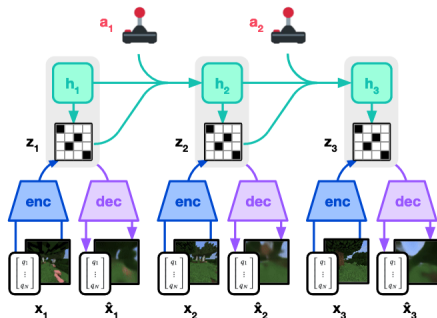
PlaNet (2019)

- Introduced the **Recurrent State-Space Model** (RSSM), which uses both deterministic and stochastic transition components for accurate multi-step predictions.
- A purely model-based agent that chooses actions via online planning **in an imagined latent space**.

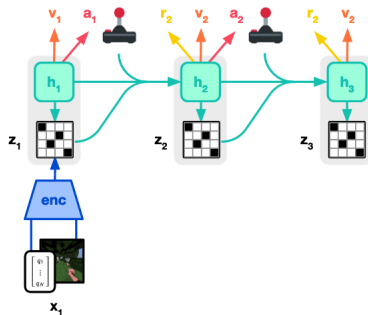
Dreamer Series

- DreamerV1 (2020) replaced the online planning with an **actor-critic** to shift the computational burden from inference-time search to offline imagination.
- DreamerV2 (2021) switched to **categorical latent spaces** and introduced KL balancing, which better captures multimodal distributions and prevents the posterior collapse.
- DreamerV3 (2023) added **robustness techniques** based on normalization, balancing, and transformations enable stable learning across domains.

Overview of Dreamer



(a) World Model Learning

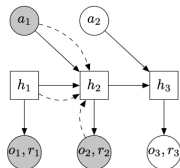


(b) Actor Critic Learning

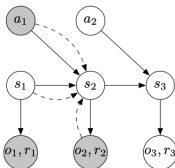
- World Model (RSSM) compresses observation inputs into latent representations and learns to predict future states based on histories and actions.
- The Actor and Critic optimize policies by learning from imagined trajectories generated by the world model.

RSSM: Learning latent dynamics in a POMDP

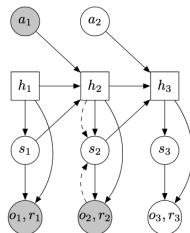
- **Deterministic Models:** cannot easily capture multiple possible futures or handle the inherent uncertainty of partial observability
- **Stochastic Models:** difficult to remember information over multiple steps.
- **Recurrent State-Space Model:** splits the latent state into two distinct parts that work together (like a sequential VAE).



(a) Deterministic model (RNN)



(b) Stochastic model (SSM)



(c) Recurrent state-space model (RSSM)

World Model Learning

Denote observation x_t , stochastic representation z_t , recurrent state h_t , model state $s_t = (h_t, z_t)$, episode continuation flag $c_t \in \{0, 1\}$, action a_t and reward r_t .

$$\text{RSSM} \left\{ \begin{array}{ll} \text{Sequence model:} & h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}) \\ \text{Encoder:} & z_t \sim q_\phi(z_t \mid h_t, x_t) \\ \text{Dynamics predictor:} & \hat{z}_t \sim p_\phi(\hat{z}_t \mid h_t) \\ \text{Reward predictor:} & \hat{r}_t \sim p_\phi(\hat{r}_t \mid h_t, z_t) \\ \text{Continue predictor:} & \hat{c}_t \sim p_\phi(\hat{c}_t \mid h_t, z_t) \\ \text{Decoder:} & \hat{x}_t \sim p_\phi(\hat{x}_t \mid h_t, z_t) \end{array} \right.$$

World Model Learning

The world model is optimized end-to-end by minimizing the prediction loss, the dynamics loss, and the representation loss:

$$\mathcal{L}(\phi) \doteq \mathbb{E}_{q_\phi} \left[\sum_{t=1}^T (\beta_{\text{pred}} \mathcal{L}_{\text{pred}}(\phi) + \beta_{\text{dyn}} \mathcal{L}_{\text{dyn}}(\phi) + \beta_{\text{rep}} \mathcal{L}_{\text{rep}}(\phi)) \right].$$

$$\mathcal{L}_{\text{pred}}(\phi) \doteq -\ln p_\phi(x_t \mid z_t, h_t) - \ln p_\phi(r_t \mid z_t, h_t) - \ln p_\phi(c_t \mid z_t, h_t)$$

$$\mathcal{L}_{\text{dyn}}(\phi) \doteq \max(1, \text{KL}[\text{sg}(q_\phi(z_t \mid h_t, x_t)) \parallel p_\phi(z_t \mid h_t)])$$

$$\mathcal{L}_{\text{rep}}(\phi) \doteq \max(1, \text{KL}[q_\phi(z_t \mid h_t, x_t) \parallel \text{sg}(p_\phi(z_t \mid h_t))])$$

View as a sequential VAE:

- $\mathcal{L}_{\text{pred}}$ is the reconstruction loss while also predicting r_t and c_t .
- The Encoder q_ϕ is the Posterior, and the Transition Model p_ϕ is the Prior.
 $\text{KL}(q_\phi \parallel p_\phi)$ is decomposed to $\mathcal{L}_{\text{dyn}}(\phi)$ and $\mathcal{L}_{\text{rep}}(\phi)$.

Actor-Critic Learning

Denote return $R_t = \sum_{\tau=0}^{\infty} \gamma_{t+\tau}^{\tau}$, prediction horizon T ; actor $a_t \sim \pi_{\theta}(a_t|s_t)$ and critic $v_{\psi}(R_t|s_t)$; predicted value expectation $v_t \doteq E[v_{\psi}(\cdot|s_t)]$.

- ① Starting from representations of replayed inputs, the world model and actor generate an imagined trajectory.
- ② Compute bootstrapped λ -returns that integrate the predicted rewards and the values:

$$R_t^{\lambda} \doteq r_t + \gamma c_t((1 - \lambda)v_t + \lambda R_{t+1}^{\lambda}), \quad R_T^{\lambda} \doteq v_T$$

- ③ Critic learns to predict the distribution of the return estimates R_t^{λ} :

$$\mathcal{L}(\psi) \doteq - \sum_{t=1}^T \ln p_{\psi}(R_t^{\lambda} | s_t)$$

- ④ Actor learns to maximize return with entropy regularizer:

$$\mathcal{L}(\theta) \doteq - \sum_{t=1}^T \text{sg}((R_t^{\lambda} - v_{\psi}(s_t)) / \max(1, S)) \log \pi_{\theta}(a_t | s_t) + \eta H[\pi_{\theta}(a_t | s_t)]$$

Categorical Latents (DreamerV2)

One-hot encodes discrete latent spaces with straight-through gradients.

Algorithm 1: Straight-Through Gradients with Automatic Differentiation

```
sample = one_hot(draw(logits))           # sample has no gradient
probs  = softmax(logits)                  # want gradient of this
sample = sample + probs - stop_grad(probs) # has gradient of probs
```

Advantages:

- 1 Preventing posterior collapse and blurry predictions.
- 2 Better capturing multi-modal futures.
- 3 Easier to optimize through straight-through gradients.
- 4 Naturally model the non-smooth, discrete events.

Symlog Transformation

Challenge: The scale of inputs and rewards can vary across domains.

Solution:

- Use the symlog function and its inverse symexp function.

$$\text{symlog}(x) \doteq \text{sign}(x) \ln(|x| + 1) \quad \text{symexp}(x) \doteq \text{sign}(x)(\exp(|x|) - 1)$$

- Apply the transformation to the loss function.

$$\mathcal{L}(\theta) \doteq \frac{1}{2}(f(x, \theta) - \text{symlog}(y))^2 \quad \hat{y} \doteq \text{symexp}(f(x, \theta))$$

Advantages:

- 1 Compress the magnitudes of both large positive and negative values.
- 2 Symmetric around the origin while preserving the input sign.
- 3 Approximates the identity around the origin so that it does not affect the learning of targets that are already small enough.

Symexp Twohot Loss

Challenge: Converting regression to classification for stochastic targets.

Solution:

- 1 The network outputs the logits for a softmax distribution over exponentially spaced bins $b_i \in B$.
- 2 The targets are two-hot encoded, which is a vector with $|B|$ entries that are all 0 except at the indices k and $k + 1$ of the two bins closest to the encoded scalar. The two entries sum up to 1, with linearly higher weight given to the bin that is closer to the encoded continuous number.
- 3 The network is then trained to minimize the categorical cross-entropy loss for classification with soft targets:

$$\mathcal{L}(\theta) \doteq - \text{twohot}(y)^T \log \text{softmax}(f(x, \theta))$$

EMA with Return Percentiles

Recall the Actor objective function:

$$\mathcal{L}(\theta) \doteq - \sum_{t=1}^T \text{sg}((R_t^\lambda - v_\psi(s_t)) / \max(1, S)) \log \pi_\theta(a_t | s_t) + \eta H[\pi_\theta(a_t | s_t)]$$

- ① Dividing by the range S to normalize into $[0, 1]$.
- ② Leave small returns below threshold $L = 1$ untouched to avoid amplifying small noise under sparse rewards.
- ③ Smooth out the outliers using an exponential moving average with return percentiles:

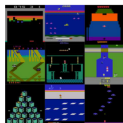
$$S \doteq \text{EMA}(\text{Per}(R_t^\lambda, 95) - \text{Per}(R_t^\lambda, 5), 0.99)$$

Benchmark Results

- Evaluate the generality of DreamerV3 across 8 domains with over 150 tasks under fixed hyperparameters.
- Baselines: Overall-tuned and fixed PPO, and other tuned expert algorithms.



(a) Control Suite



(b) Atari



(c) ProcGen

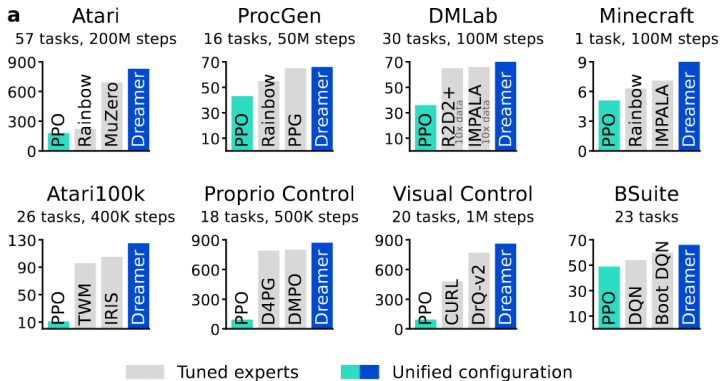


(d) DMLab



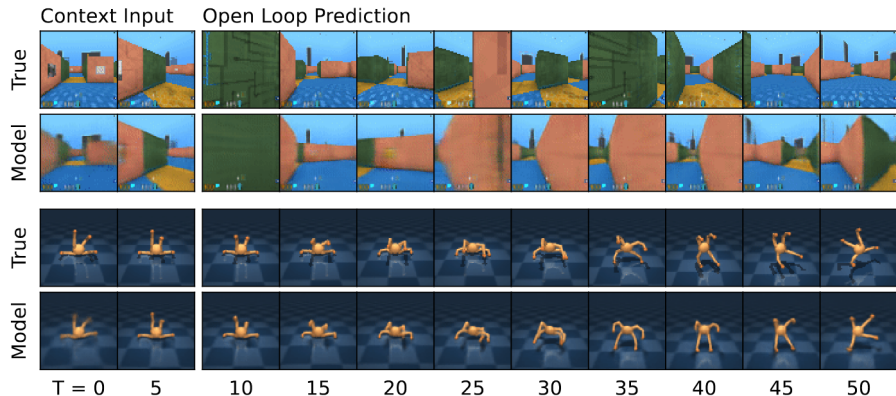
(e) Minecraft

Benchmark Results



Dreamer outperforms tuned expert algorithms and high-quality implementation of the widely applicable PPO across a wide range of benchmarks and data budgets.

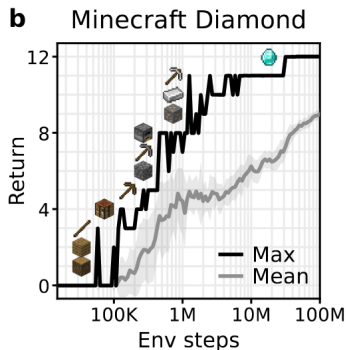
Multi-step Video Prediction



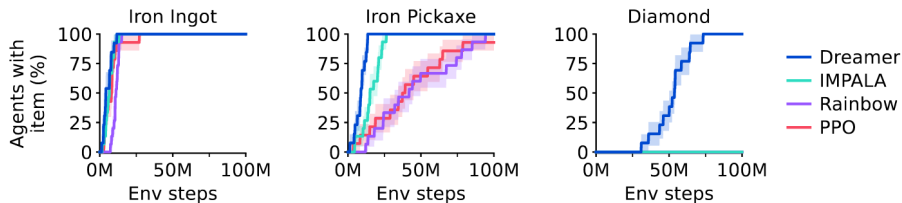
The world model learns an understanding of the underlying structure of each environment.

Minecraft Diamond Task

- Autonomously learning to collect diamonds in Minecraft without human data has been widely recognized as a substantial challenge because of the sparse rewards, exploration difficulty, long time horizons, and the procedural diversity of this open-world game.
- Dreamer learns to obtain diamonds in Minecraft from scratch, for which previous approaches required human data or domain-specific heuristics.



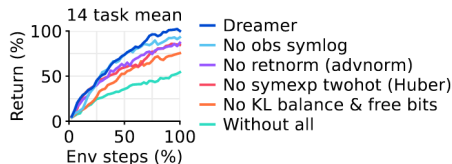
Minecraft Diamond Task



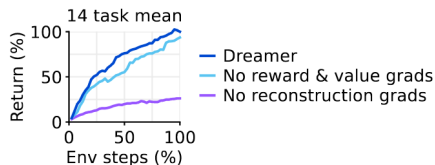
Although previous algorithms progress up to the iron pickaxe, Dreamer is the only compared algorithm that manages to discover a diamond, and does so reliably.

Ablations Study

a Robustness techniques



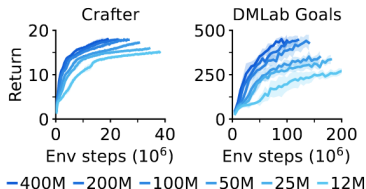
b Learning signals



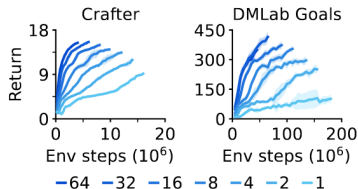
- All individual robustness techniques contribute to the performance of Dreamer on average.
- The performance of Dreamer predominantly rests on the unsupervised reconstruction loss, instead of reward and value prediction.

Robust Scaling

c Model size scaling



d Replay scaling



- larger models not only increase task performance but also require less environmental interaction.
- Higher replay ratios predictably increase the performance of Dreamer.

Conclusion

- **Universal Mastery:** First RL algorithm to solve 150+ tasks (Atari, Minecraft, Robotics) using fixed hyperparameters—no per-task tuning required.
- **A Minecraft Milestone:** The first agent to collect diamonds from scratch, proving it can handle extreme sparse rewards and long-term planning.
- **Robust Scaling:** Performs consistently across varying data and compute budgets, making it a practical tool for real-world applications.

Is it worth reading? Yes.

- Establishes the Dreamer series as the definitive blueprint for World Models.
- Directly aligns with the current AI trend that moves toward generalist agents.

Is it worth implementing? Yes.

- One of the most robust MBRL algorithms across diverse domains.
- The architecture is uniquely suited for patient trajectory modeling.